MONITOR Z · L15.6

Clase 6 — Cómo comparten los datos: DSECT, área común y reentrancia

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L15 · Clase 6 — Cómo comparten los datos: DSECT, área común y reentrancia

La Clase 4 dejó una pregunta abierta: si `ZTKFMT` está en otra CSECT, no ve las etiquetas de ZTKMON. ¿Cómo llega entonces a `WJOBID` o a `NREP`? La respuesta es una técnica que **ZTKMON ya usa**, aunque todavía no la viste con nombre: describir los datos con una **DSECT** y llegar a ellos por un **registro** que apunta a la memoria donde están.

> 📚 Fuentes: el listado del ensamblado de **JOB34775**: la DSECT `ENTRADA` (sentencias 10116–10123), `USING ENTRADA,R9` (10147), las tablas `NUEVA` y `VIEJA` (13004–13005), sentencias 10579–10581, 11032, 12181–12198, y `PRTDCB` (12830). El *SAVE OPERATION SUMMARY* del binder del mismo job. *HLASM V1R6 Language Reference* (SC26-4940): DSECT, USING con etiqueta. *z/OS MVS Programming: Assembler Services Guide* (SA23-1368): «Reentrant programs».

1. El problema: los datos viven adentro del programa

- Hoy los 2.036 bytes de datos de trabajo (`NREP`, `VUELTA`, `WJOBID`, fechas…) están **adentro de la CSECT ZTKMON**,

entre X'1DDC' y X'25D0' (L15, punto 2). Ocupan lugar de las bases.

- Todas las rutinas los nombran por su etiqueta, porque están en la misma CSECT.
- Con módulos, eso se rompe: una etiqueta de ZTKMON no existe para `ZTKFMT`.

2. Una DSECT es un molde sin memoria

Una **DSECT** (*dummy section*) describe la **forma** de un bloque de datos (qué campo hay, en qué desplazamiento y de qué largo), **sin reservar memoria**. ZTKMON tiene una suya (sentencias 10117–10123):

Loc	Fuente	Qué es
000000	`ENTRADA DSECT`	el molde de **un job** de las tablas: X'38' = 56 bytes
+000	`EJOBID DS CL8`	el jobid

+008	`EJOBN DS CL8`	el nombre
+010	`EOWNR DS CL8`	el dueño
+018	`ECLAS DS CL8`	la clase
+020	`ETSINI DS CL19`	cuándo empezó

- Los `Loc` empiezan en 0 y **no hay bytes en la columna de objeto**: el ensamblador no genera nada. Sólo aprende que

«`EJOBN` está a +8 del comienzo, sea donde sea el comienzo».

- Las macros de IBM son lo mismo: `IAZSSST DSECT=YES`, `CVT DSECT=YES`, `IHAASCB DSECT=YES` … describen bloques de JES y de

z/OS que **ya existen** en memoria.

Analogía: una DSECT es una **plantilla transparente** con casilleros marcados. No tiene datos propios: la apoyás sobre cualquier pedazo de memoria y te dice qué es cada byte.

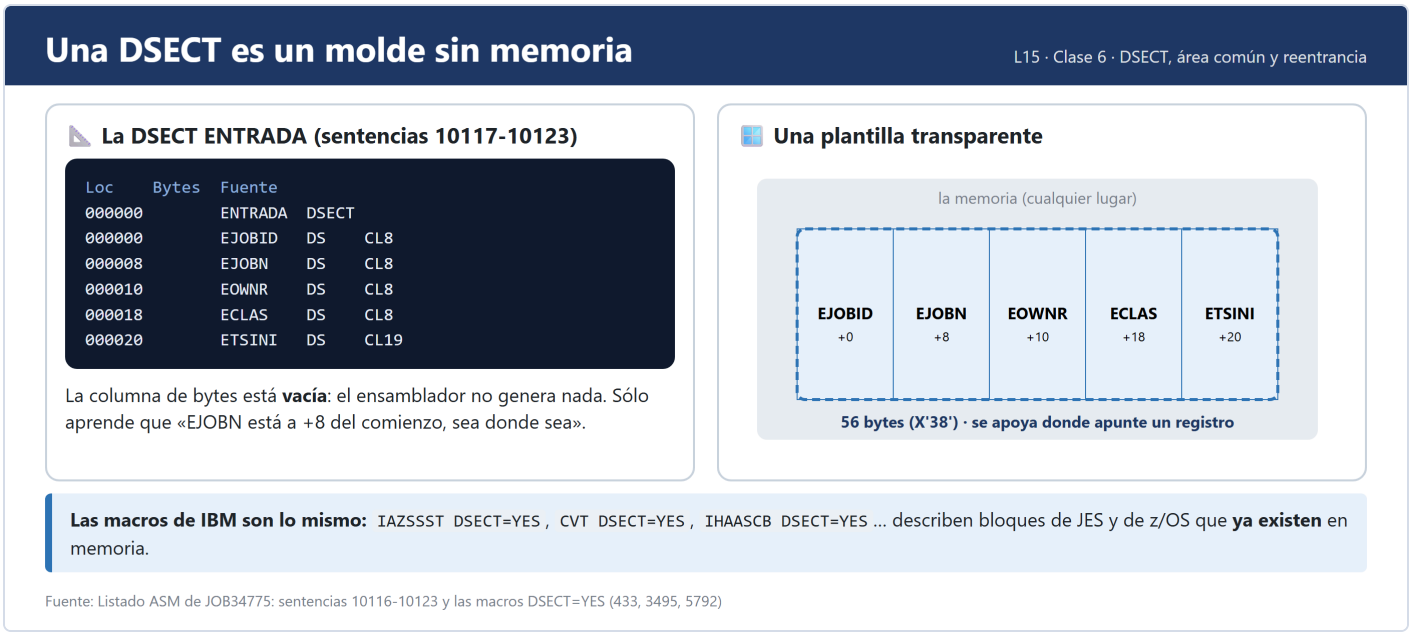


Lámina L15-22_dsect_molde

3. Cómo se usa: `USING` molde, registro

Para que la plantilla diga algo hay que **apoyarla** sobre una dirección. Eso hacen un `USING` y un registro:

Sent.	Fuente	Bytes	Qué hace
10147	`QEN USING ENTRADA,R9`	(no genera bytes)	promesa: «R9 apunta a un `ENTRADA`»
10579	`L R9,=A(NUEVA)`	`5890 A650`	R9 = la dirección de la tabla `NUEVA`
10581	`MVC QEN.EJOBID,QTR.STTRJID`	`D207 9000 800C`	copia el jobid de JES a `EJOBID`: **base R9, desplazamiento 000**

- Es **base + desplazamiento** (Clase 3), pero la base no es uno de los tres pisos: es R9, que apunta adonde vos quieras.
- Para pasar al job siguiente alcanza con sumarle 56 a R9: la misma plantilla sirve para las 200 entradas.

El dato clave: `NUEVA` está en X'2880' y `VIEJA` en X'5440'. Cada una mide 200 × 56 = **11.200 bytes** (`MAXJOB EQU 200`, `ENTLEN EQU 56`). Van de X'2880' a X'8000': casi todo **fuera de los tres pisos**. Por eso la CSECT mide X'F72C' y las bases cubren sólo X'3000': lo que se usa por un registro propio **no ocupa lugar de las bases**.

Un detalle que conviene saber: los «1.920 bytes libres» de la L15 son el tramo X'2880'–X'3000', y ahí es donde **empieza** `NUEVA`. Cuentan como libres porque ninguna instrucción nombra ese tramo por los pisos: `NUEVA` se usa por R9. Si el código crece, `NUEVA` y `VIEJA` se corren más arriba, y nada se rompe.

ZTKMON usa la misma técnica con los bloques de IBM: `USING SSOB,R2`, `USING CVT,R1`, `USING ASCB,R6`…



Lámina L15-23_using_sobre_registro

4. El área común: la misma técnica para los datos de trabajo

La propuesta de la L15 es hacer con los datos de trabajo lo mismo que ya se hace con las tablas:

1. **El molde:** una DSECT `ZTKCOM` con todos los campos compartidos (`NREP`, `VUELTA`, `WJOBID`…), en un miembro `COPY`

que incluyen todos los módulos. Así todos tienen la misma plantilla.

1. **La memoria:** el principal la pide **una sola vez** con `STORAGE OBTAIN`, como ya pide el anillo (`PIDEANI`, sentencias

12181–12198): `STORAGE OBTAIN,LENGTH=(2),LOC=31,COND=YES` y después `ST R1,ANILLO`. El anillo quedó en X'19B00000' (JOB34879, Clase 2).

1. **La dirección:** viaja en un registro fijo (por ejemplo `USING ZTKCOM,R9`) o en la lista de parámetros de cada `CALL`

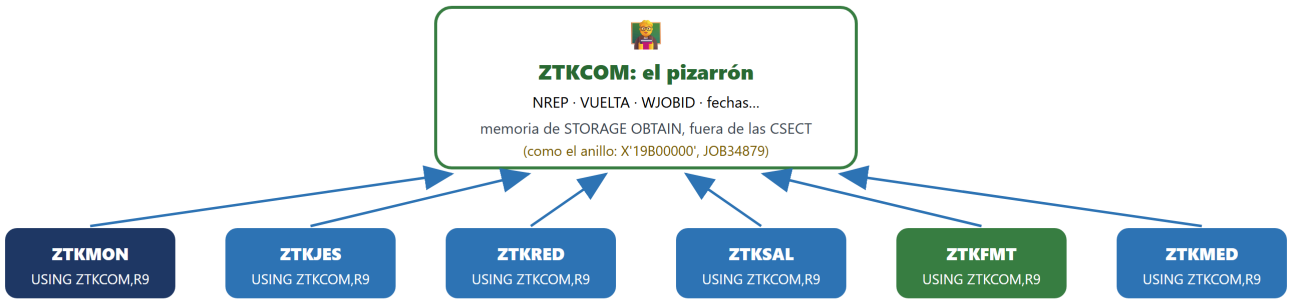
(Clase 5).

Analogía: en vez de que cada oficina tenga su propia copia del pizarrón, hay **un solo pizarrón** en la sala común, y todos saben en qué sala está.

Lo que se gana: los ~2.036 bytes de datos salen de las bases, y los pisos quedan para código.

El área común: un solo pizarrón para todos

L15 · Clase 6 · DSECT, área común y reentrancia



#	Paso	Cómo	Ya existe en ZTKMON
1	📐 El molde	una DSECT `ZTKCOM` en un miembro `COPY` que incluyen todos los módulos	`ENTRADA DSECT`
2	🧱 La memoria	el principal la pide **una vez**: `STORAGE OBTAIN,…,LOC=31`	`PIDEANI` → `ST R1,ANILLO` (12198)
3	📍 La dirección	en un registro fijo (`R9`) o en la lista de parámetros del `CALL`	`L R9,=A(NUEVA)`

Lo que se gana: los ~2.036 bytes de datos salen de las bases, y los pisos quedan para código.

Fuente: Listado ASM de JOB34775: PIDEANI (12181-12198), ENTRADA (10117) y 10579 · +DIAG ANILLO de JOB34879

Lámina L15-24_area_comun

5. Reentrancia: que el programa no se escriba a sí mismo

Un programa es **reentrante** si **no modifica sus propios bytes** mientras corre. Así, una sola copia en memoria puede servir a varias tareas a la vez, y z/OS puede guardarla en memoria protegida. La L07 lo pide para que ZTKMON sea un STC de verdad.

ZTKMON hoy no es reentrante, y el listado lo muestra:

Testigo	Qué dice
`ST R1,NREP` (sentencia 11032) = `5010 BE2C`	escribe en X'1E2C': **adentro del propio programa**
`NUEVA` y `VIEJA` en X'2880' y X'5440'	las tablas que se reescriben en cada vuelta están adentro de la CSECT
`SAVEAREA DS 18F` en X'1F88' (Clase 5)	la *save area* también está adentro: cada `CALL` escribe ahí

binder de JOB34775	`RENT NO`, `REUS NO`, `REFR NO`: el módulo no se marcó reentrante, y no lo es

Cómo se arregla con el área común: si los datos, las tablas y las *save areas* pasan a memoria pedida con `STORAGE OBTAIN`, el código deja de escribirse a sí mismo. Con eso se puede marcar `RENT`.

El cuidado: si un módulo marcado `RENT` todavía escribe en sí mismo y z/OS lo carga en memoria protegida, esa escritura termina en un abend. Hay que sacar **todo** lo que se escribe, no sólo una parte.

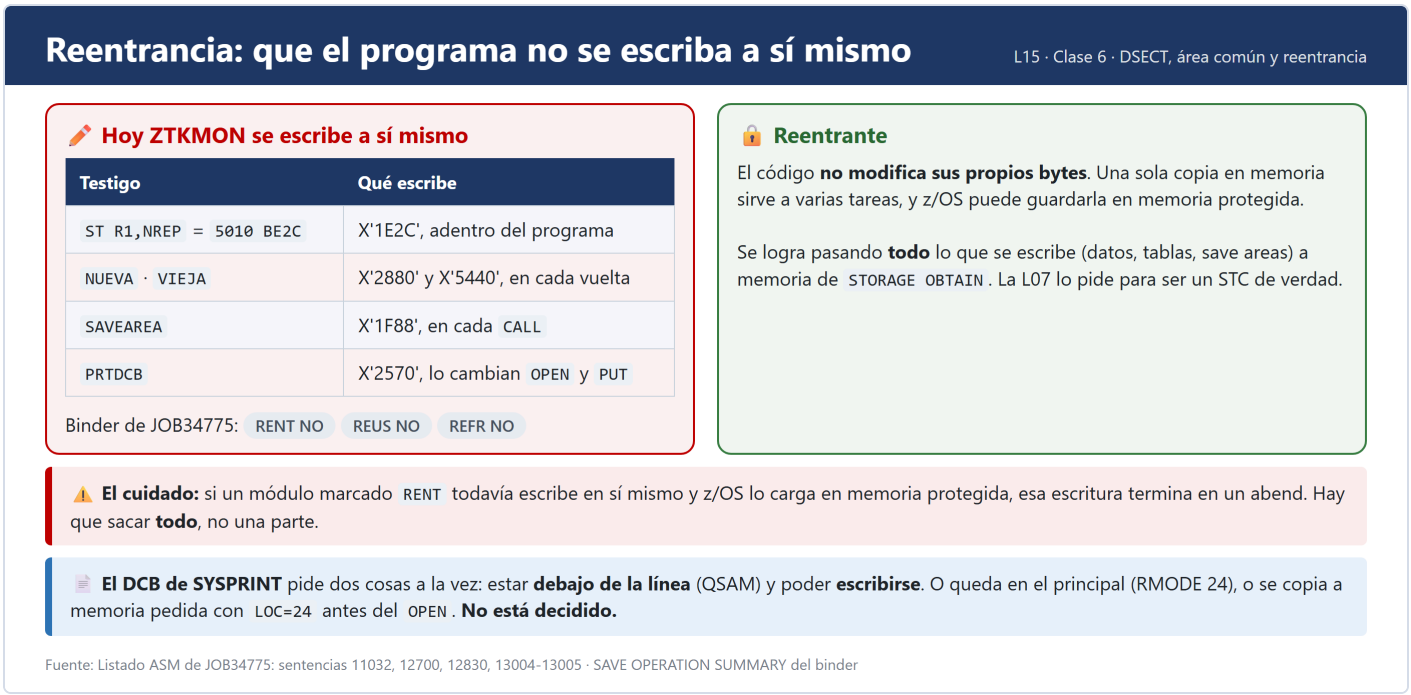


Lámina L15-25_reentrancia

6. El caso especial: el `DCB` de SYSPRINT

- `PRTDCB DCB DDNAME=SYSPRINT,…` está en **X'2570'**, adentro de la CSECT (sentencia 12830).
- Pide dos cosas a la vez:
  - **Estar debajo de la línea de 16 MB:** QSAM lo exige. Por eso ZTKMON es `RMODE 24` (Clase 2).
  - **Que se pueda escribir:** `OPEN` y `PUT` modifican el `DCB`. Mientras esté adentro del programa, el programa no es

reentrante.

- **Las dos salidas posibles:** que el `DCB` quede en el principal (que sigue `RMODE 24`) aceptando que esa parte no es

reentrante, o que se copie a memoria pedida con `LOC=24` antes del `OPEN`. Se decide al escribir el código; **no está decidido**.

7. Qué cambia con los módulos

	Hoy	Con `ZTKCOM`
Dónde están los datos	adentro de ZTKMON, entre X'1DDC' y X'25D0'	en memoria de `STORAGE OBTAIN`, fuera de todas las CSECT
Cómo los nombra una rutina	por etiqueta (`NREP`)	por la plantilla: `QCOM.NREP`, con `USING ZTKCOM,R9`
Quién los ve	sólo ZTKMON	**todos** los módulos que tengan la dirección
Reentrante	no (`RENT NO`)	sí, si también salen las tablas, las *save areas* y el `DCB`

8. Resumen

Idea	En una línea
DSECT	un molde: dice qué campo está en qué desplazamiento, sin reservar memoria
`USING molde,registro`	apoya el molde donde apunta el registro; base + desplazamiento con una base libre
Ya lo usás	`ENTRADA` sobre R9 para las tablas `NUEVA` / `VIEJA` (fuera de los tres pisos), y los bloques de IBM
Área común	una DSECT `ZTKCOM` compartida, memoria pedida una vez, la dirección en un registro
Reentrante	el programa no escribe en sí mismo; hoy ZTKMON sí escribe (`ST R1,NREP`, `RENT NO`)
`DCB`	tiene que estar debajo de la línea y se escribe: hay que decidir dónde va

9. Preguntas de comprobación

1. `ENTRADA DSECT` empieza en `Loc` 000000 y no genera ningún byte. Entonces, cuando el programa hace `MVC QEN.EJOBID,QTR.STTRJID` (`D207 9000 800C`), ¿dónde están de verdad los bytes que escribe?
2. ¿Por qué `ST R1,NREP` hace que ZTKMON no sea reentrante? ¿Qué cambia si `NREP` pasa a `ZTKCOM`, sobre memoria pedida con `STORAGE OBTAIN`?

Mis respuestas

Notas